

Flask + MySQL

学生管理系统

接口文档 v1.0

适用对象：Web开发课程学生
版本日期：2026-04-28

接口概览

本接口文档描述学生管理系统的所有API端点，供前后端开发参考使用。

接口名称	请求方法	URL	说明
学生列表	GET	/	显示所有学生
添加学生	GET/POST	/add	添加新学生
编辑学生	GET/POST	/edit/<id>	编辑指定学生
删除学生	GET	/delete/<id>	删除指定学生

1. 学生列表接口

基本信息

字段	值
接口名称	学生列表
请求方法	GET
URL	/
返回格式	HTML页面

后端代码

```
@app.route('/') def index(): """■■ - ■■■■■■""" students = Student.query.all() # ■■■■■■
return render_template('index.html', students=students)
```

2. 添加学生接口

基本信息

字段	值
接口名称	添加学生
请求方法	GET/POST
URL	/add
Content-Type	application/x-www-form-urlencoded

请求参数

参数名	类型	必填	说明
name	string	是	学生姓名，最大50字符
age	integer	是	学生年龄，范围1-150

后端代码

```
@app.route('/add', methods=['GET', 'POST']) def add_student(): if request.method == 'POST': name = request.form.get('name') age = request.form.get('age') # 添加学生 student = Student(name=name, age=age) db.session.add(student) db.session.commit() flash(f'添加成功 {name}', 'success') return redirect(url_for('index')) return render_template('add.html')
```

3. 编辑学生接口

基本信息

字段	值
接口名称	编辑学生
请求方法	GET/POST
URL	/edit/<student_id>
参数类型	path（整数）

后端代码

```
@app.route('/edit/', methods=['GET', 'POST']) def edit_student(student_id): student = Student.query.get_or_404(student_id) if request.method == 'POST': student.name = request.form.get('name') student.age = request.form.get('age') db.session.commit() flash('■■■■■■■■■■', 'success') return redirect(url_for('index')) return render_template('edit.html', student=student)
```

4. 删除学生接口

基本信息

字段	值
接口名称	删除学生
请求方法	GET
URL	/delete/<student_id>
注意	建议使用POST或添加确认机制

后端代码

```
@app.route('/delete/') def delete_student(student_id): student =
Student.query.get_or_404(student_id) db.session.delete(student) db.session.commit() flash(f'■■■
{student.name} ■■■', 'success') return redirect(url_for('index'))
```

5. 数据库模型

Student 模型结构

字段名	类型	约束	说明
id	Integer	PK, AUTO_INCREMENT	主键
name	String(50)	NOT NULL	学生姓名
age	Integer	NOT NULL	学生年龄
created_at	DateTime	DEFAULT NOW()	创建时间

模型代码

```
class Student(db.Model): __tablename__ = 'students' # ■■■ id = db.Column(db.Integer,
primary_key=True, autoincrement=True) name = db.Column(db.String(50), nullable=False) age =
db.Column(db.Integer, nullable=False) created_at = db.Column(db.DateTime, default=datetime.now)
def __repr__(self): return f''
```

6. 错误处理

HTTP状态码	含义	处理方式
200	请求成功	正常返回页面或数据
302	重定向	表单提交后跳转
404	资源不存在	显示404页面
500	服务器错误	显示500错误页面

提示：实际生产环境中，建议使用 POST 方法处理删除操作，以防止CSRF攻击和误操作。